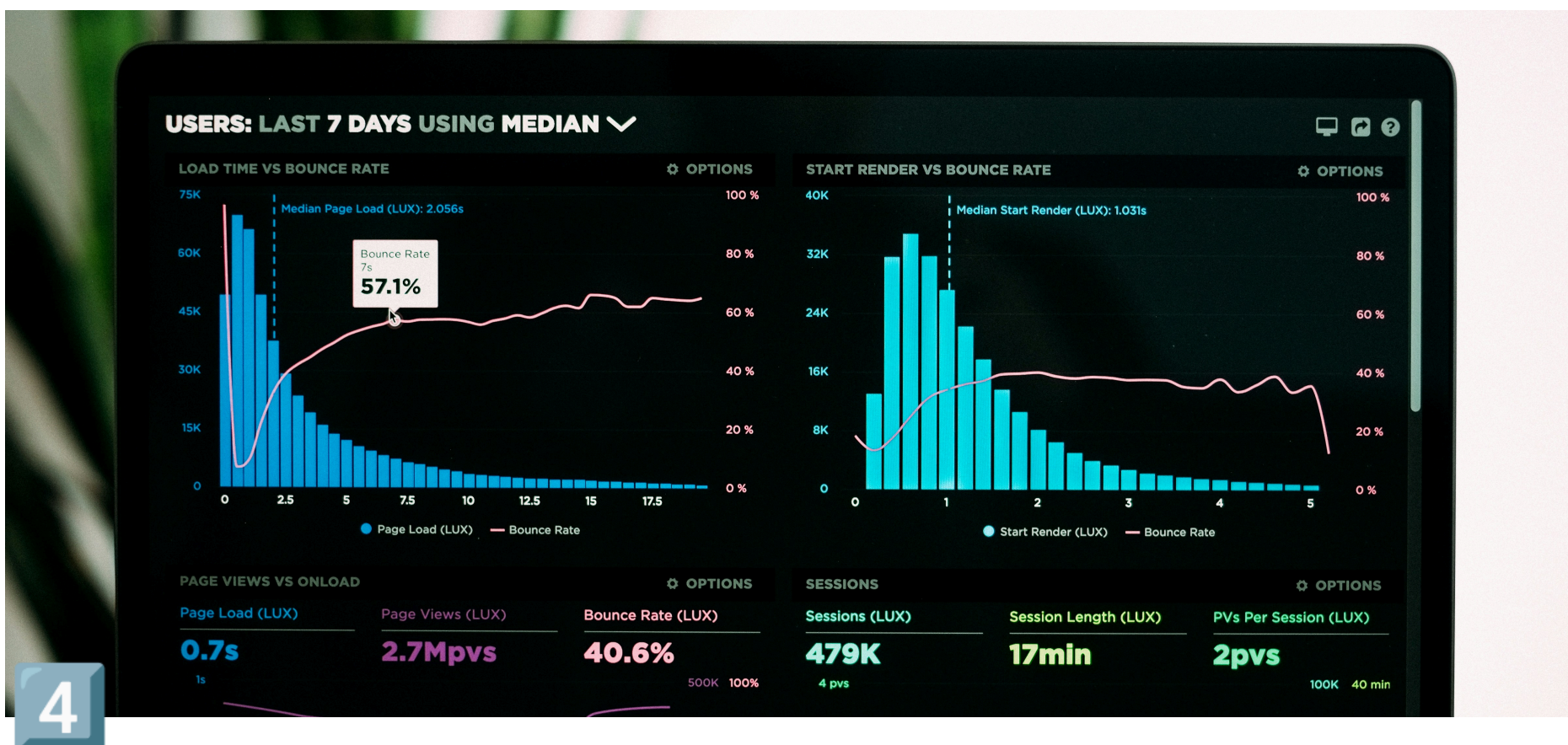




ЛР 4. Loki + Prometheus + Grafana

▼ Задача

Подключить к Airflow и Spark сбор метрик + логирование. Осуществить визуализацию через Grafana

▼ Гайд

▼ Часть 1. Логирование

1. Проверить, что папки с логами *airflow* и *spark* монтируются извне!
2. Добавить **Loki** и **Alloy** в композ файл (можно локально, можно через git репозиторий с автодеплоем):

```
...
loki: # сервис-обработчик логов
  image: grafana/loki:3.2.0
  container_name: loki # имя контейнера Loki, важно для последующих шагов
  ports:
    - "3100:3100"
  command: -config.file=/etc/loki/local-config.yaml # запуск с дефолтным конфигом

alloy: # сервис-сборщик логов
  image: grafana/alloy:v1.5.1
  volumes:
    - ./logs:/opt/logs # здесь `./logs` - это локальная директория, которая монтируется в /opt/airflow/logs! Может называться по-другому
    - ./alloy.conf:/etc/alloy/config.alloy # локальное имя можно любое
  command:
    - run
    - --server.http.listen-addr=0.0.0.0:12345
    - /etc/alloy/config.alloy

grafana: # сервис визуализации
  image: grafana/grafana:11.2.0
  ports:
    - "3000:3000"
  environment:
    - GF_AUTH_ANONYMOUS_ENABLED=true
    - GF_AUTH_ANONYMOUS_ORG_ROLE=Admin
```

```
- GF_PATHS_PROVISIONING=/etc/grafana/provisioning
```

```
...
```

3. Создать файл `alloy.conf` (или любое другое имя, главное чтоб совпадало с прописанным в композе) со следующим содержанием:

```
local.file_match "airflow_logs" {
  path_targets = [
    {"__path__" = "/opt/logs/dag_id=*/run_id=*/task_id=*/*.log", "job" = "airflow_logs"} // по маске ищем нужные логи
  ]
}

loki.source.file "airflow" {
  targets      = local.file_match.airflow_logs.targets
  forward_to   = [loki.write.default.receiver] // отправка собранных логов в дефолтный ресивер Loki
}

loki.write "default" { // дефолтный ресивер Loki
  endpoint {
    url = "http://loki:3100/loki/api/v1/push" // адрес может отличаться (container_name)
  }
}
```

4. Запускаем композ-проект, затем осуществляем хотя бы один прогон своего DAG, чтоб в примонтированной папке с логами появились файлы, которые “ждет” сервис Alloy. Структура будет примерно такая: `папка_с_логами/dag_id=xxxx/run_id=xxxx/task_id=xxxx.log`

```
llidd@lenid MINGW64 ~/itmo/airflow-spark-example (main)
$ ls -la logs/
total 4
drwxr-xr-x 1 llidd 197609 0 Apr 29 01:54 .
drwxr-xr-x 1 llidd 197609 0 Apr 29 01:46 ..
drwxr-xr-x 1 llidd 197609 0 Apr 29 01:54 'dag_id=my_dag'
drwxr-xr-x 1 llidd 197609 0 Apr 29 01:46 dag_processor_manager
drwxr-xr-x 1 llidd 197609 0 Apr 29 01:46 scheduler

llidd@lenid MINGW64 ~/itmo/airflow-spark-example (main)
$ ls -la logs/dag_id=my_dag/
total 0
drwxr-xr-x 1 llidd 197609 0 Apr 29 01:54 .
drwxr-xr-x 1 llidd 197609 0 Apr 29 01:54 ..
drwxr-xr-x 1 llidd 197609 0 Apr 29 01:54 'run_id=scheduled__2026-04-27T22:53:53.877422+00:00'

llidd@lenid MINGW64 ~/itmo/airflow-spark-example (main)
$ ls -la logs/dag_id=my_dag/run_id=scheduled__2026-04-27T22:53:53.877422+00:00/
total 0
drwxr-xr-x 1 llidd 197609 0 Apr 29 01:54 .
drwxr-xr-x 1 llidd 197609 0 Apr 29 01:54 ..
drwxr-xr-x 1 llidd 197609 0 Apr 29 01:54 'task_id=test'

llidd@lenid MINGW64 ~/itmo/airflow-spark-example (main)
$ ls -la logs/dag_id=my_dag/run_id=scheduled__2026-04-27T22:53:53.877422+00:00/task_id=test/
total 8
drwxr-xr-x 1 llidd 197609 0 Apr 29 01:54 .
drwxr-xr-x 1 llidd 197609 0 Apr 29 01:54 ..
-rw-r--r-- 1 llidd 197609 4719 Apr 29 01:54 'attempt=1.log'

llidd@lenid MINGW64 ~/itmo/airflow-spark-example (main)
$
```

5. Проверить в логах Alloy, что он не только запустился, но и “подцепил” все нужные log-файлы:

```

llidd@lenid MINGW64 ~/itmo/airflow-spark-example (main)
$ docker logs --tail 10 f55dab7cd7dd
ts=2026-04-28T22:51:44.432956736Z level=info msg="finished node evaluation" controller_path=/ controller_id="" trace_id=e80098d71bf64bef62e85c659ddbe31b node_id=loki.write.default duration=376.903µs
ts=2026-04-28T22:51:44.43301619Z level=info msg="finished node evaluation" controller_path=/ controller_id="" trace_id=e80098d71bf64bef62e85c659ddbe31b node_id=local.file_match.airflow_logs duration=47.206µs
ts=2026-04-28T22:51:44.43332136Z level=info msg="finished node evaluation" controller_path=/ controller_id="" trace_id=e80098d71bf64bef62e85c659ddbe31b node_id=loki.source.file.airflow duration=297.15µs
ts=2026-04-28T22:51:44.433348424Z level=info msg="finished complete graph evaluation" controller_path=/ controller_id="" trace_id=e80098d71bf64bef62e85c659ddbe31b duration=1.236363ms
ts=2026-04-28T22:51:44.433829027Z level=info msg="scheduling loaded components and services"
ts=2026-04-28T22:51:44.43478721Z level=info msg="starting cluster node" service=cluster peers_count=0 peers="" advertise_addr=127.0.0.1:12345
ts=2026-04-28T22:51:44.435023215Z level=info msg="peers changed" service=cluster peers_count=1 peers=f55dab7cd7dd
ts=2026-04-28T22:51:44.435961458Z level=info msg="now listening for http traffic" service=http addr=0.0.0.0:12345
ts=2026-04-28T22:54:14.450068955Z level=info msg="Seeked /opt/logs/dag_id=my_dag/run_id=scheduled__2026-04-27T22:53:53.877422+00:00/task_id=test/attempt=1.log - 8{Offset:0 Whence:0}" component_path=/ component_id=loki.source.file.airflow
ts=2026-04-28T22:54:14.450121755Z level=info msg="tail routine: started" component_path=/ component_id=loki.source.file.airflow component=tailer path="/opt/logs/dag_id=my_dag/run_id=scheduled__2026-04-27T22:53:53.877422+00:00/task_id=test/attempt=1.log"

llidd@lenid MINGW64 ~/itmo/airflow-spark-example (main)
$ █

```

▼ Часть 2. Сбор метрик

1. Добавить в `docker-compose.yml` сервис **Prometheus**:

```

...
prometheus:
  image: prom/prometheus:v2.48.1
  container_name: prometheus # имя контейнера, важно для последующих шагов
  ports:
    - "9090:9090"
  volumes:
    - ./prometheus.yml:/etc/prometheus/prometheus.yml # конфиг-файл, где описываем наши target-ы
  command:
    - '--config.file=/etc/prometheus/prometheus.yml'
...

```

2. Добавить файл с конфигом Прометее, который будем “прокидывать”

▼ `prometheus.yml`

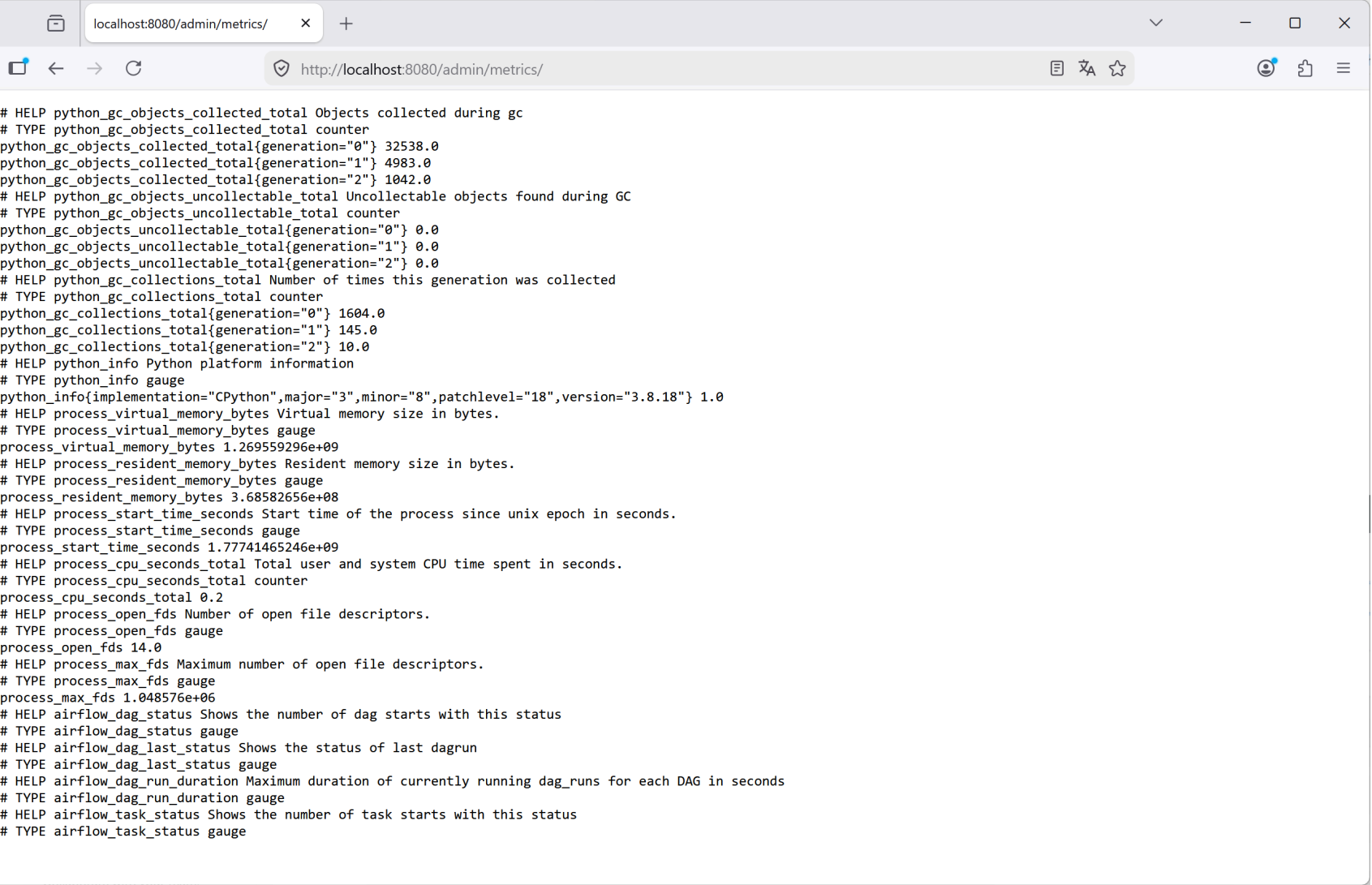
```

global:
  scrape_interval: 10s # частота сбора

scrape_configs:
  - job_name: 'airflow' # по этому имени потом будем "искать" нужный индекс
    metrics_path: /admin/metrics/
    static_configs:
      - targets: ['airflow-webserver:8080'] # хост (имя контейнера) и порт

```

3. Рестартим compose проект, проверяем что Airflow отдает метрики:



4. Заходим на localhost:9090, проверяем что Прометей взлетел и видит метрики со стороны Airflow (*Status* → *Targets*):



Prometheus

Alerts

Graph

Status ▾

Help

Targets

All scrape pools ▾

AllUnhealthyCollapse All

🔍

Filter by endpoint or labels

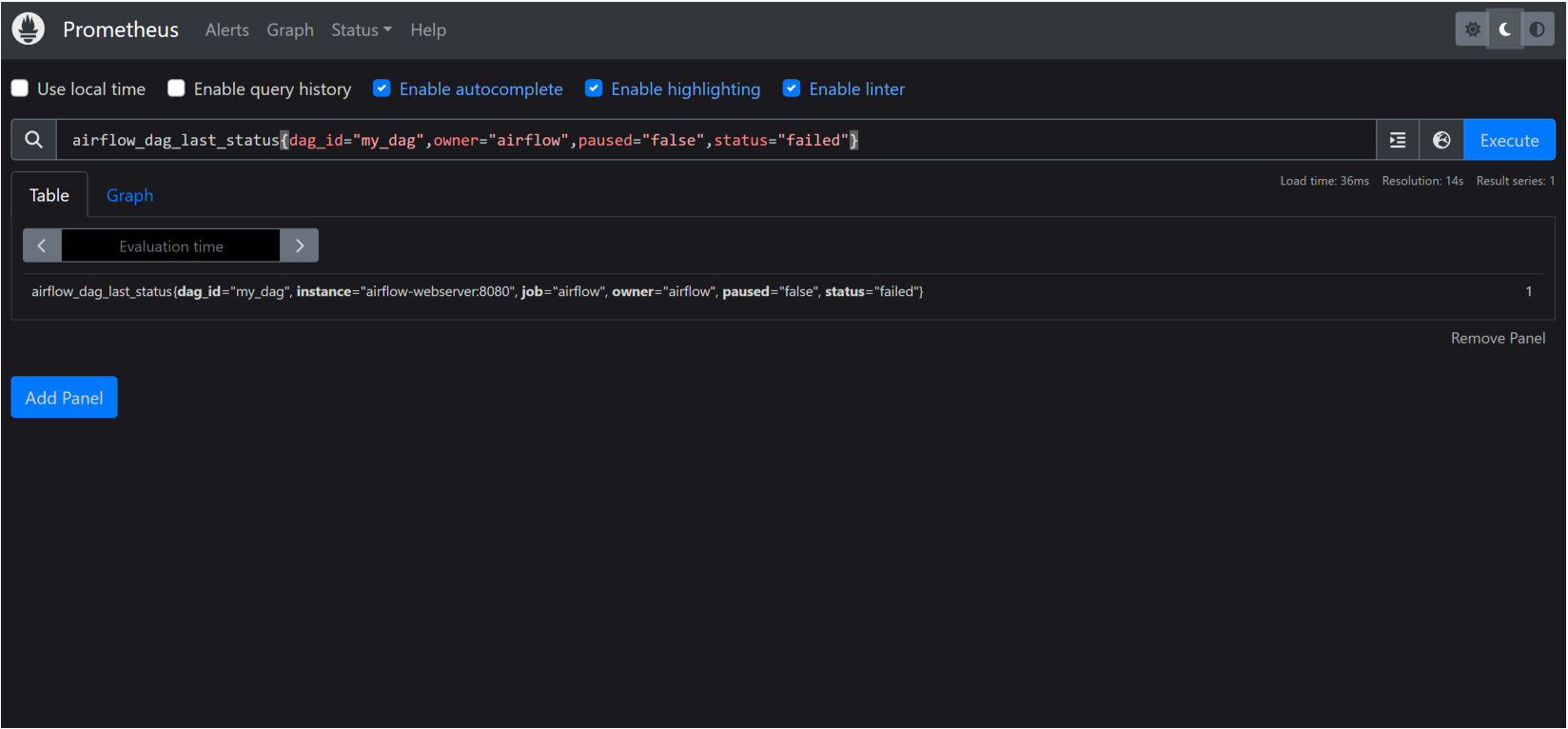
airflow (1/1 up)

show less

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://airflow-webserver:8080/admin/metrics/	UP	instance="airflow-webserver:8080" job="airflow" ⌵	4.78s ago	86.103ms	

- Запускаем DAG, проверяем что на `/admin/metrics` появились новые метрики - их в том числе можно посмотреть прямо в Прометее, сделав поиск по имени метрики в разделе Graph

▼ Пример для поиска неуспешных run-ов:

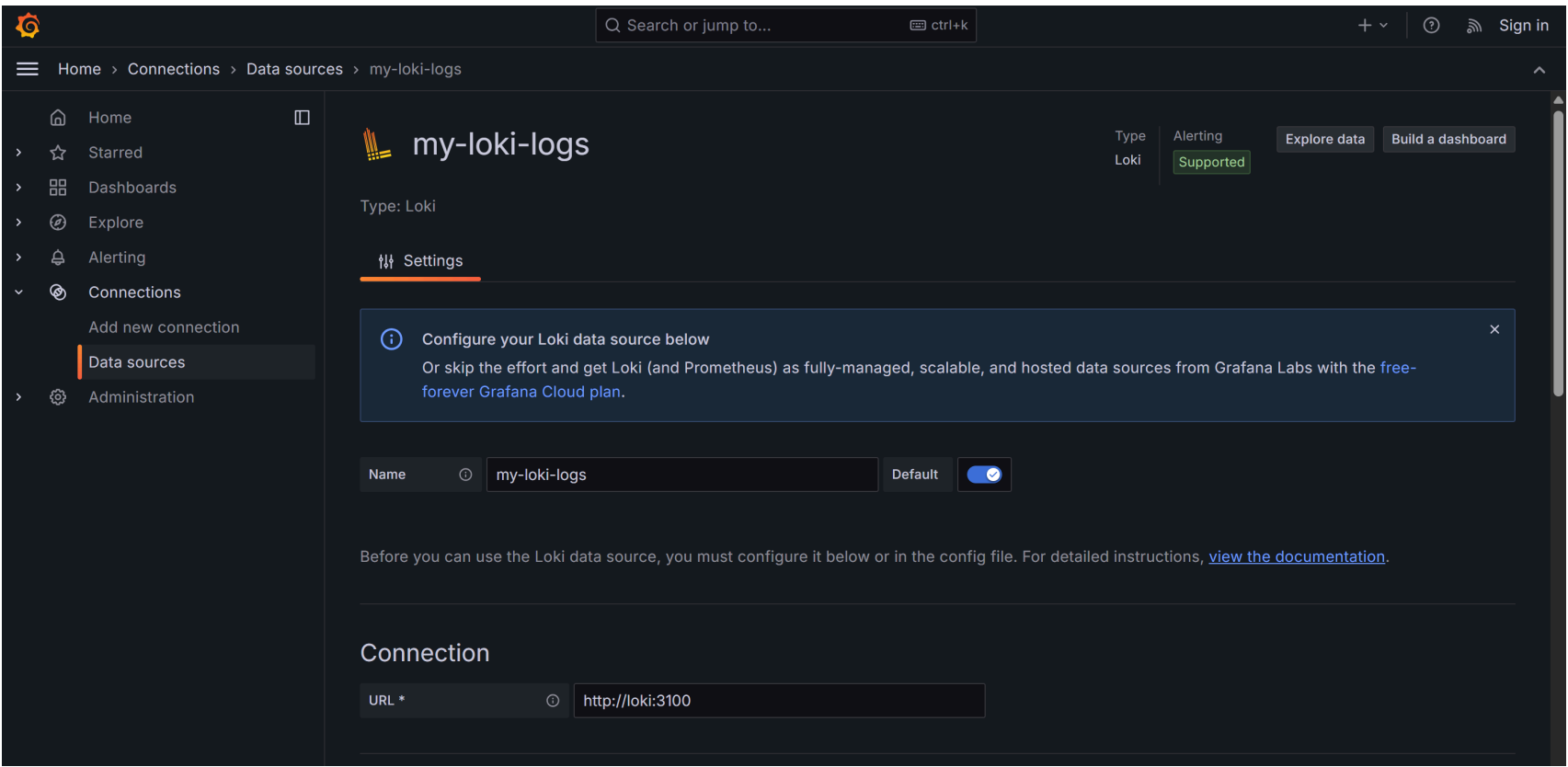


▼ Часть 3. Визуализация

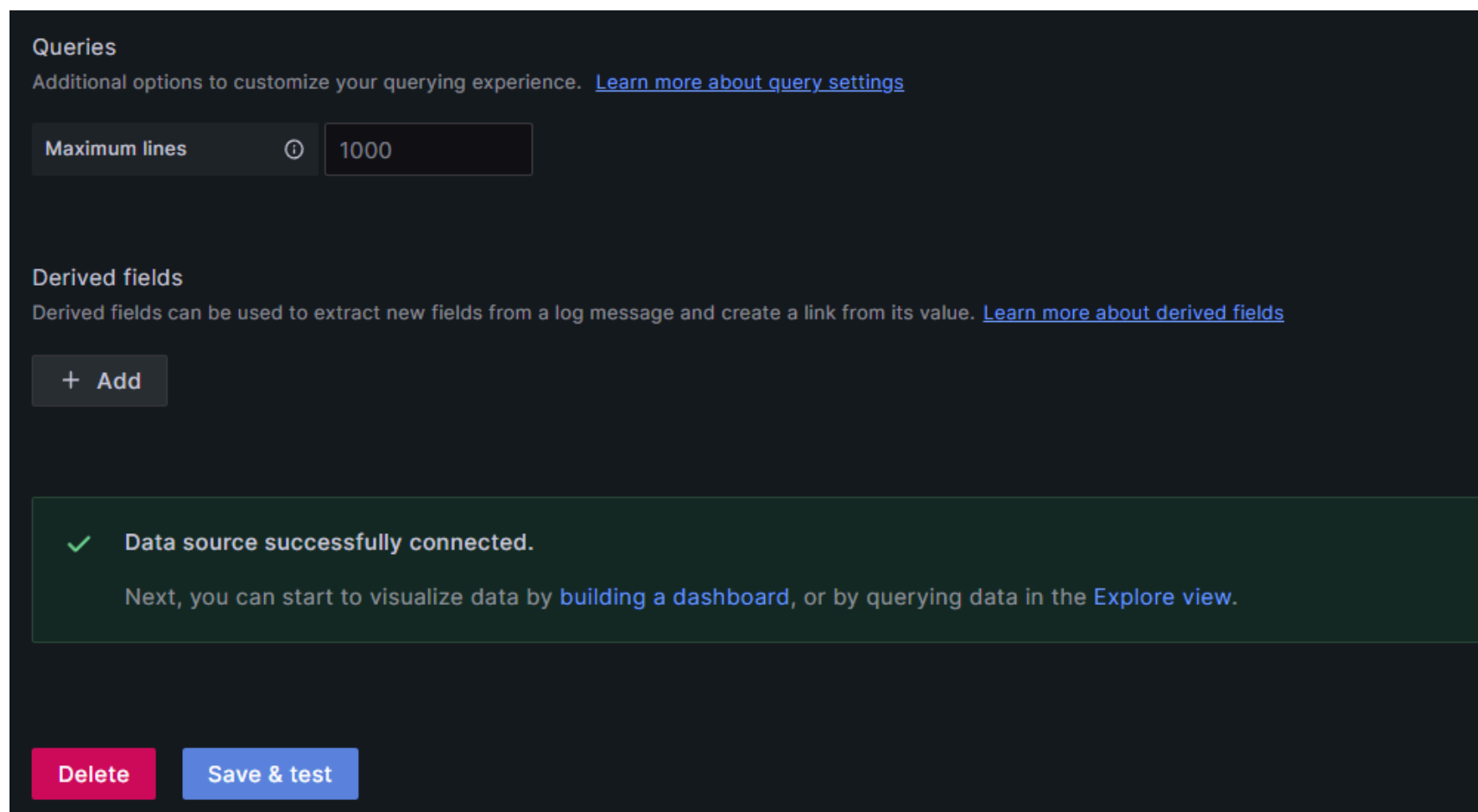
1. В композ добавляем финальную часть - сервис **Grafana**. Рестартируем проект

```
...
grafana: # сервис визуализации
  image: grafana/grafana:11.2.0
  ports:
    - "3000:3000"
  environment:
    - GF_AUTH_ANONYMOUS_ENABLED=true
    - GF_AUTH_ANONYMOUS_ORG_ROLE=Admin
    - GF_PATHS_PROVISIONING=/etc/grafana/provisioning
...
```

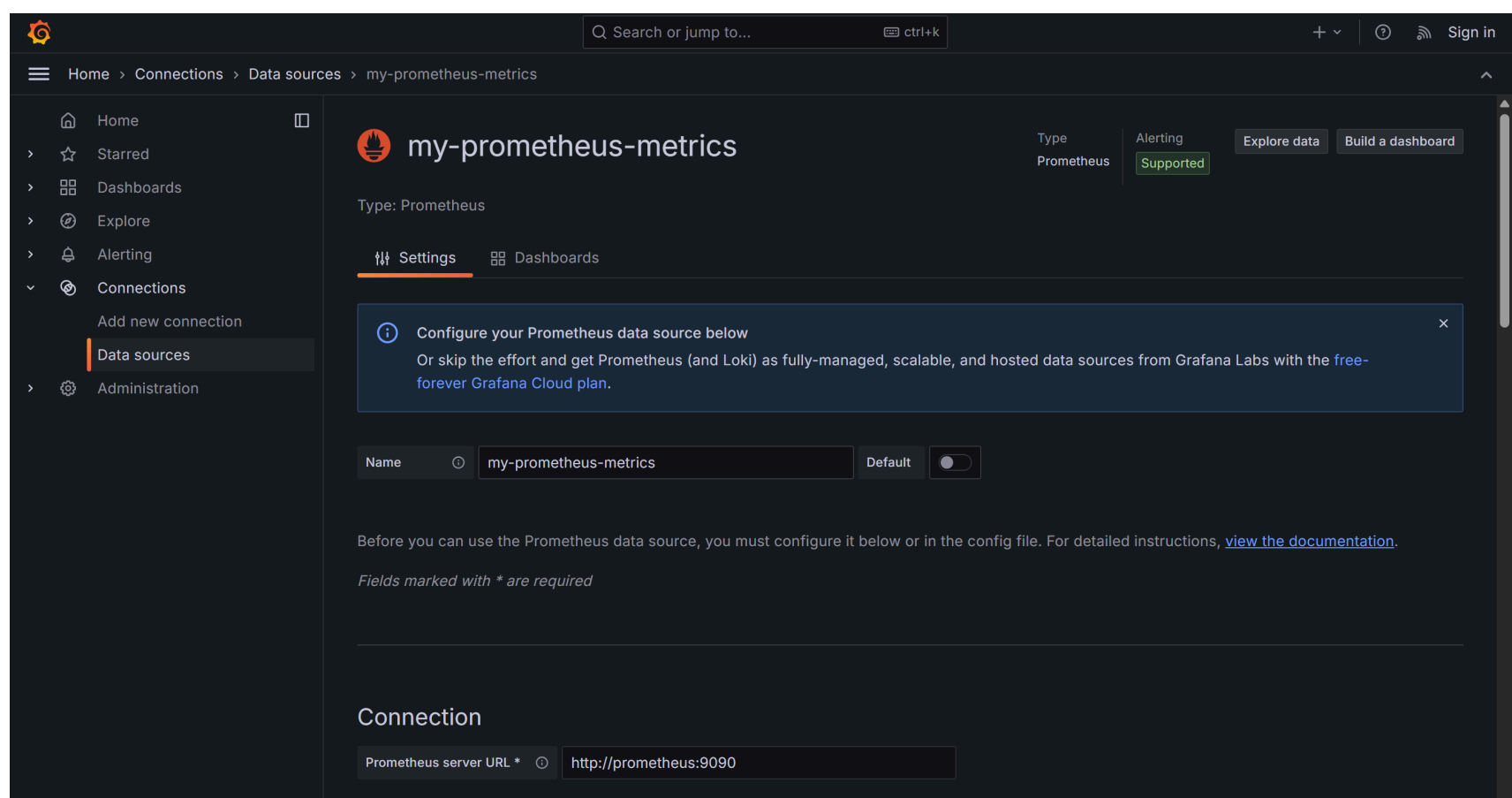
2. Заходим в Графану по `http://localhost:3000`. Подключаем Loki к Grafana, раздел *Connections* → *Data sources* → *Loki*. В настройках подключения указать любое имя и нужный адрес (например `http://loki:3100`), все остальное можно оставить по дефолту:



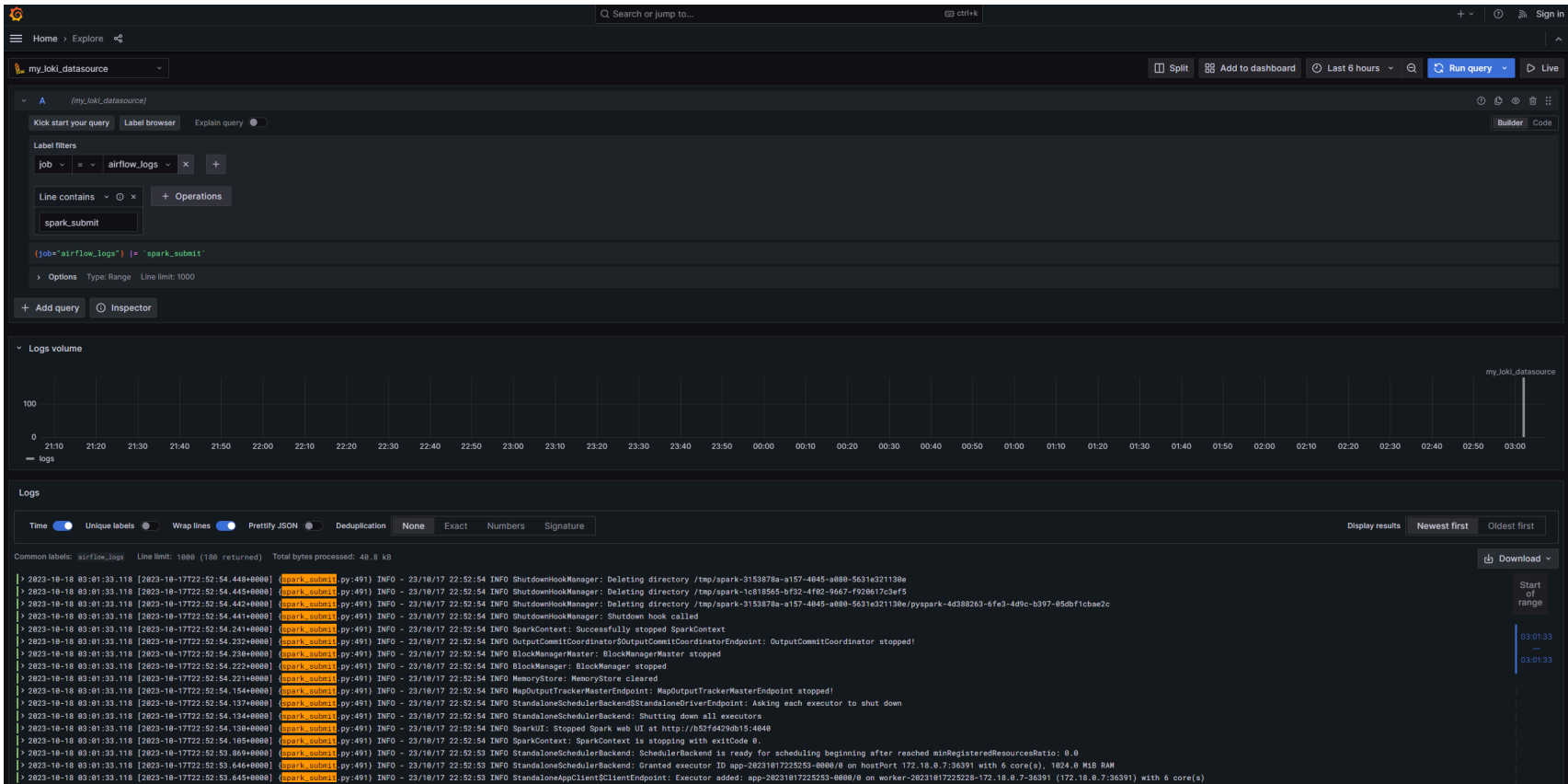
3. Сохранить подключение, нажав **Save & Test**. Если нет ошибок и сервис предлагает перейти к визуализации и/или просмотру данных, значит в Части 1 все настроено правильно



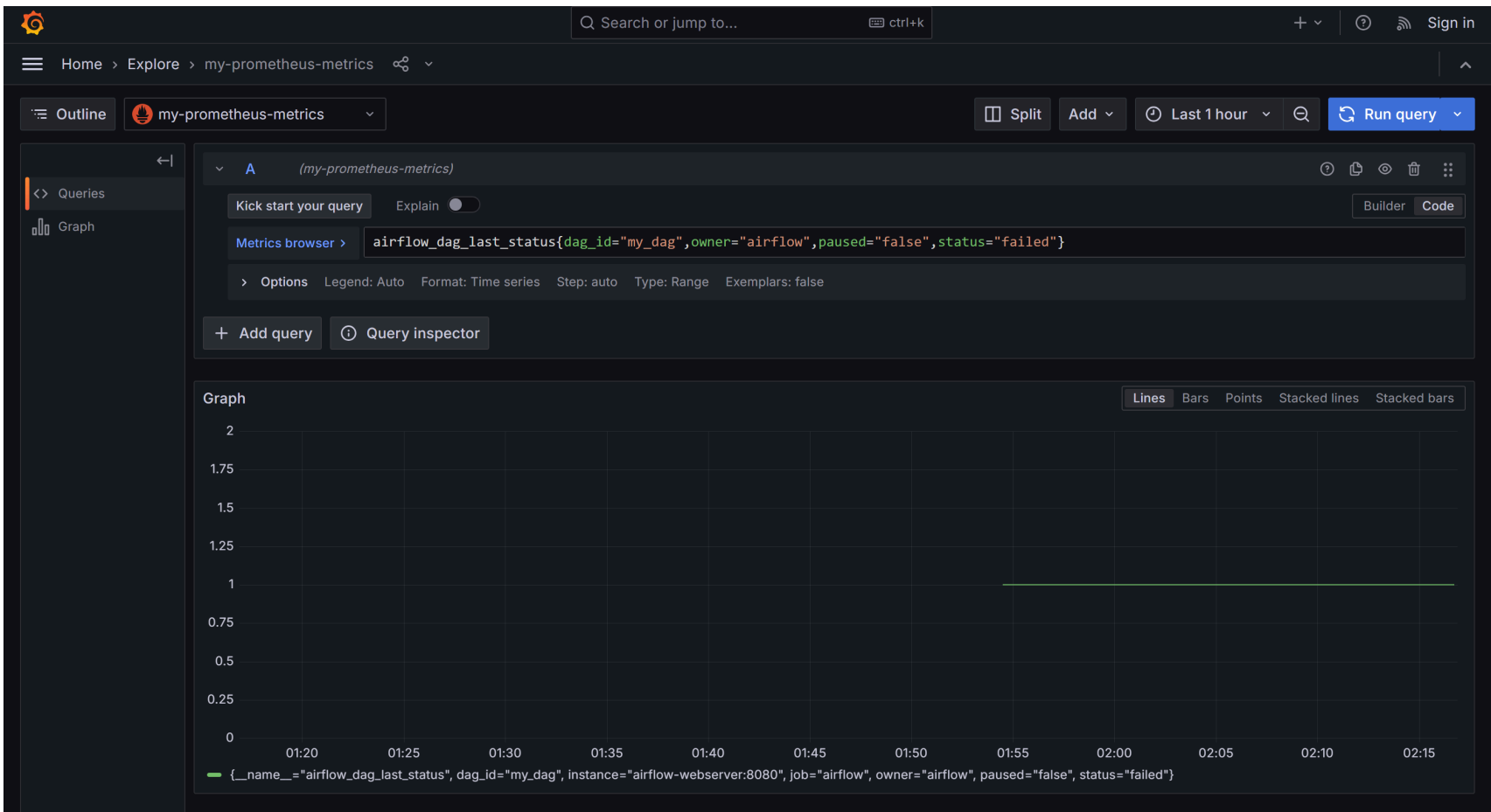
4. Точно так же подключаем новый датасурс, в этот раз Prometheus. URL указываем согласно композ-файлу, например `http://prometheus:9090` :



5. Теперь можно перейти в *Explore* (на этой же странице или через общее меню), выбрать датасурс Loki, затем выбрать в качестве селектора (индекса) имя конкретного файла ИЛИ имя джобы/индекса с шага 3 Части 1. Затем задать какой-нибудь фильтр (например, поиск подстроки `spark_submit`), нажать *Run query* и увидеть свои логи



6. То же самое с Прометеем, при указании конкретной метрики:

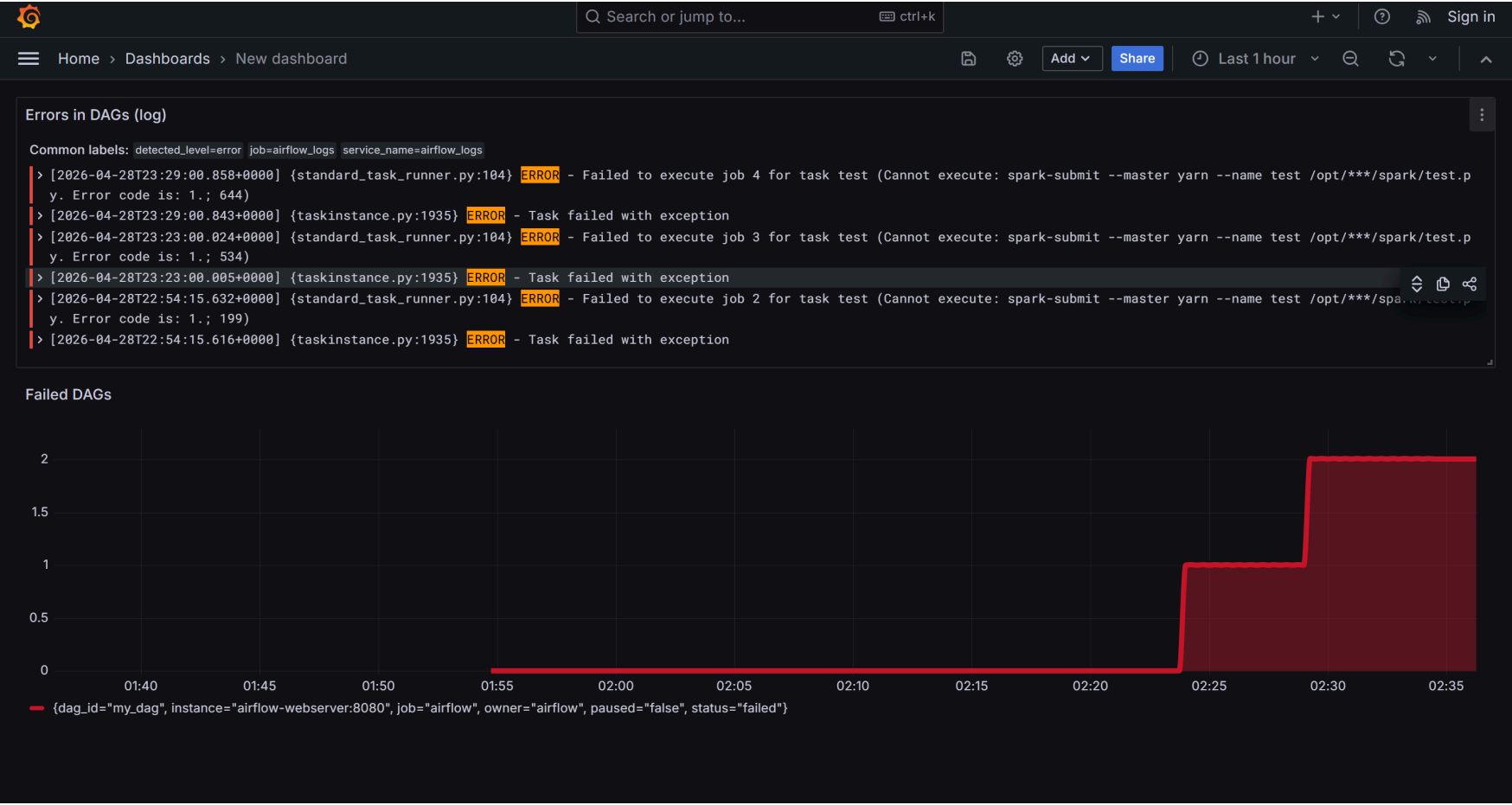


7. Оба query можно сохранять и добавлять в виде плитки на дашборды (*Add → Add to dashboard*). Там же можно красиво визуализировать - распарсить логи по колонкам, покрасить значения метрик в какой-то цвет, etc.

▼ Задание

Необходимо сделать следующее:

- Подключить сбор логов в Loki **И/ЛИ** экспорт метрик в Prometheus для сервиса Spark
 - Создать дашборд с двумя простыми плитками в Grafana с использованием созданных ранее датасурсов.
- Пример:



▼ Отчетность

- Скриншот с Grafana
- Новые конфиги Alloy/Prometheus, если есть